



ICS3U Resource Book

Revised August 2013

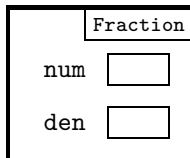
6.1 Creating Objects

In mathematics, a fraction can be defined as an expression of the form $\frac{a}{b}$ together with rules about how we perform operations such as addition and multiplication on such expressions. Examples of fractions are expressions like $\frac{3}{4}$, $\frac{7}{-3}$, or $\frac{-2}{5}$. In the first case, we are thinking of the *class* of all fractions; in the second we are thinking of particular *instances* of that class. In Java, to help us manipulate things like fractions (along with much more complex things), we can construct *classes* that define the way that things look and behave along with *objects* that give us instances of those classes.

As an example of these classes and objects, let us look at how we might introduce the concept of a fraction into Java. Ignoring, for the moment, the operations that can be performed on fractions, we can consider a fraction to be composed of two parts: a numerator and a denominator. If we want the numerator and denominator of our fractions to both be of type `int`, we can define the class of fractions as follows:

```
class Fraction
{
    int num;
    int den;
}
```

This class is very different from the ones that we have seen up to this point in our study of Java. It does not have a `main` method; in fact, it has no methods at all! It simply defines what something called a `Fraction` should look like: something that has two `int` parts that we have called a `num` and a `den`. We call these parts *fields* of the class. A field is declared *outside* a method while a local variable is declared *inside* a method.¹ We can illustrate the structure of the class with a diagram, as follows:



¹We should warn you that, although we think that this distinction between fields and local variables is a useful one, not everybody uses these terms.

We can think of the `Fraction` class as a model, blueprint, or template of what a fraction should look like. All fractions have both a numerator and a denominator. The class definition makes this idea explicit; it defines a new type called `Fraction` that we can use in our programs. The definition does not actually create any fractions; it simply shows us what any fractions that we eventually create will look like. When we do create any fractions, each one of them will have its own `num` and `den` fields. Since each instance of a fraction will have these fields, they are, reasonably enough, called *instance fields*. In a program that uses fractions, this class can appear separately from the class that contains the `main` method.

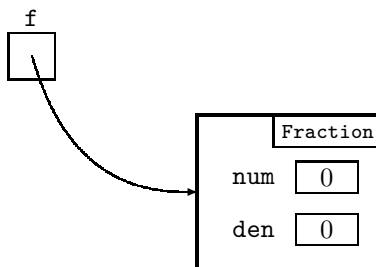
Once we have defined what we mean by a fraction, we can then create actual fractions, using our template. To begin to do this, we can, in the `main` method of our program, say, make the declaration:

```
Fraction f;
```

This creates a variable `f` that can act as a *reference* to a fraction. Since we have not assigned `f` a value, it does not at this point actually refer to a fraction. To achieve this, we can now write:

```
f = new Fraction();
```

This causes Java to create an *object*, an instance of the type `Fraction`. In addition a reference to the location in memory of the newly created object is stored in the variable `f`. Pictorially, we can illustrate what we have created as follows:



Notice in the preceding diagram that the `num` and `den` fields are both shown with the value zero. Although Java does not initialize local variables when we declare them, it does initialize all numeric fields to the value zero when we create an object containing those fields. We can (and usually will) combine the process of declaring and creating a new `Fraction` object `f` by writing

```
Fraction f = new Fraction();
```

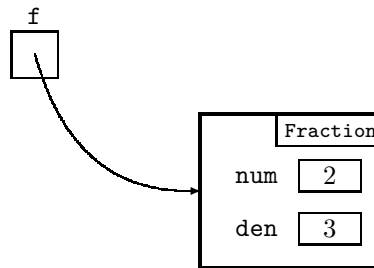
If we wanted our object to represent the fraction $\frac{2}{3}$ rather than $\frac{0}{0}$, we could now make the following assignment statements:

```
f.num = 2;  
f.den = 3;
```

You might find it useful to read these assignment statements as

```
f's num = 2;  
f's den = 3;
```

Pictorially, the result of the assignments is:



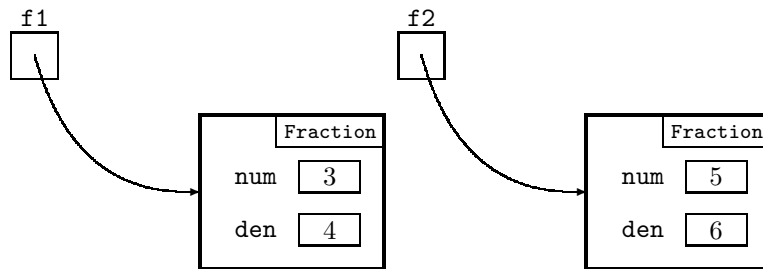
Using the same template, we can create as many objects of that type as we wish.

Example 1

To create two fractions, `f1` and `f2`, representing the fractions $\frac{3}{4}$ and $\frac{5}{6}$, we could write:

```
Fraction f1 = new Fraction();  
f1.num = 3;  
f1.den = 4;  
Fraction f2 = new Fraction();  
f2.num = 5;  
f2.den = 6;
```

Pictorially, the result of all of this would be:



Once we have created objects, we can manipulate their fields in the same ways that we do with local variables.

Example 2

Suppose that `f1` and `f2` have been created as shown in the previous example. If we were to write

```
f1.num--;
f1.den = f2.den + 3;
```

then `f1` would represent the fraction $\frac{3-1}{6+3} = \frac{2}{9}$ and `f2` would be unchanged.

As we have already noted, if we simply declare a reference variable, that variable does not yet actually refer to an object. For example, if we were to write

```
Fraction f1;
```

the result is an uninitialized `Fraction` variable. An attempt to use `f1` in this state will result in an error during compilation of the program. To avoid such compilation errors, we can initialize a reference variable so that it does not refer to an object (but is still defined) by assigning it the value `null`. If we write

```
Fraction f2 = null;
```

then we can print `f2` (producing the output “`null`”) or compare `f2` to other objects (an idea that we will explore in more detail later). The variables `f1` and `f2` are illustrated in the next diagram. The dot inside `f1` indicates

that it has no value. We show the `null` reference of `f2` with the symbol used to indicate a ground in a diagram of an electrical circuit.



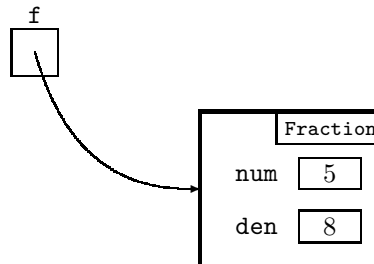
As we have already seen with strings, objects are very different from primitive types so, when we attempt to apply operations to objects, we must be very careful of what we are doing. Otherwise, the results can be something quite different from what we want. The next example examines the meaning of assignment with objects.

Example 3

Suppose that we have created an object `f` of our `Fraction` class by writing

```
Fraction f = new Fraction();
f.num = 5;
f.den = 8;
```

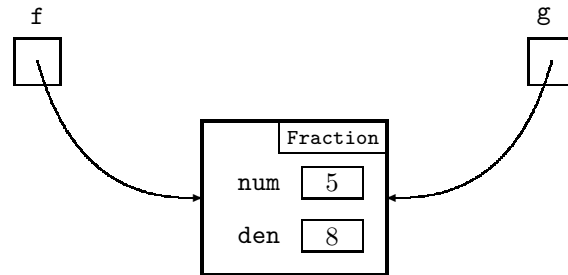
to obtain the object shown in the next diagram.



If we now write

```
Fraction g = f;
```

the result is the usual one produced by an assignment: a value in one memory location is copied into another memory location. Here, however, the copying is done from the reference `f` to the reference `g`, producing the result shown in the next diagram where `f` and `g`, since they have the same value, both refer to the same object.



If we were now to write

```
g.num = 1;
```

this would change the `num` field of the object to which `g` refers. Since `f` refers to the same object, `f` would now represent the fraction $\frac{1}{8}$. We say that `g` is an *alias* of `f`.

Exercises 6.1

1. What is the essential difference between a local variable and a field?
2. Write statements that could be used to create an object of type `Fraction` representing the fraction $\frac{1}{5}$.
3. Assuming that the class `Fraction` is defined as it was in the text, state the error in the following fragment.

```
Fraction p;
p.num = 7;
p.den = 8;
```

4. Draw diagrams like those shown in the text to illustrate the results of the given fragment.

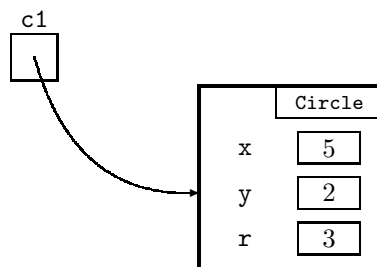
```
Fraction p, q, r;
p = new Fraction();
q = new Fraction();
r = q;
p.num = p.den = 2;
```

```
q.num = 2*p.den;  
p.den++;  
--p.num;  
r.den = p.num + 2;
```

5. Assuming that two objects **f1** and **f2** of type **Fraction** have been created and assigned values, write statements to perform each task.
- (a) Double the value of **f1**.
 - (b) Invert **f2**.
 - (c) Make **f1** equal to the (unsimplified) product of **f1** and **f2**.
 - (d) Make **f2** equal to the (unsimplified) sum of **f1** and **f2**.
 - (e) Make **f1** equal to $|f1|$.
6. A circle in the Cartesian plane can be described uniquely by its centre and its radius. Thus, a class called **Circle** that could be used to represent such circles might start with the following:

```
class Circle  
{  
    double x;    // x-coordinate of centre  
    double y;    // y-coordinate of centre  
    double r;    // radius  
}
```

An object of this class, representing a circle with centre at (5, 2) and having radius 3 is shown in the next diagram.



Using this class, write a Java program that performs the following actions.

- (a) Create `Circle` objects `c1` and `c2` where `c1` represents a circle with centre at $(1, 2)$ having radius 4 while `c2` represents a circle with centre at $(-2, 0)$ having radius 2.
 - (b) Set the `double` variable `distance` to the distance from the origin to the centre of `c1` and then print this value.
 - (c) Set the `double` variable `centreSeparation` equal to the distance between the centres of `c1` and `c2` and then print this value.
 - (d) Set the `double` variable `minDistance` to the minimum distance from a point on `c1` to a point on `c2` and then print this value.
7. A complex number $z = a + bi$ can be specified by its real part, a , and its imaginary part, b .
- (a) Create a class `Complex` that could be used to represent complex numbers.
 - (b) Create objects `z1` and `z2` that represent the complex numbers $2 + 3i$ and $5 - 4i$.
 - (c) Write a fragment that assigns to `z1` the sum of `z1` and `z2`.

6.2 Instance Methods

Objects are more than just collections of data; they can also have functionality, implemented through methods. The methods associated with objects can be grouped into two primary categories. In this section, we examine the more commonly used type — *instance methods*. We have already encountered instance methods in the `String` class where the methods `equals` and `compareTo` are of this type. Here we will examine the structure of such methods and learn how to create our own.

As an example of an instance method, suppose that we want to extend our `Fraction` class with a method that returns, as a `double` value, the magnitude or size of a fraction. We define the size of a fraction $\frac{a}{b}$ as $|\frac{a}{b}|$. To implement such a method, we could add the method shown in the next example to our `Fraction` class.

Example 1

The following instance method will return the magnitude of a `Fraction` object.

```
public double size ()
{
    return Math.abs((double)num/den);
}
```

If, in our `main` method, we had created and defined a `Fraction` object `f`, then we could determine its size, `s`, by writing

```
double s = f.size();
```

In line with our suggestion in Section 6.1, it may be helpful if you read this statement as

```
s = f's size
```

There are many things to note here.

1. The method does not have the modifier `static` that has appeared in every method that we have created up to now. Any method without the `static` modifier is an *instance* method; any method having the `static` modifier is a *class* method. We will examine the use of class methods with objects later in the chapter.
2. The general form of a call to an instance method that returns a value is
`<object identifier>.<method identifier>(<parameter list>);`
In our example, the call has the form `f.size()`;
3. In the method, we compute the size of the fraction using the identifiers `num` and `den`, not `f.num` and `f.den`. No object is mentioned explicitly in the method. Instead, it is *implicit* here that the `num` and `den` that we are referring to are those of the object that is currently associated with the method. The call to the method associates the object `f` as an *implicit parameter* of the method. If we were to call the method with a statement like

```
t = g.size();
```

then `g` would be the implicit parameter and Java would use the `num` and `den` fields of `g` in its calculations.

4. The current implicit object in an instance method can be referred to in Java as `this`. It would have been correct to write the statement in the body of the method as


```
return Math.abs((double)this.num/this.den);
```

 if we wanted to make explicit the fact that we are taking the quotient of the current object's `num` and `den` fields but there is, in this example, no need to do so. (The next example will illustrate a practical use of `this` in an instance method.)

Although the parameter list of the method in the previous example was empty, it is possible (and common) for instance methods to have non-empty parameter lists. We have already encountered this in the methods of the `String` class. There, the header of the `equals` method has the form

```
public boolean equals (String s)
```

while that of the `compareTo` method has the form

```
public int compareTo (String s)
```

To show how we can create an instance method with a non-empty parameter list, suppose that we wanted a method to compare the sizes of two `Fraction` objects.

Example 2

The following method uses the `size` method to compare two objects of type `Fraction`. It returns a reference to a `Fraction` object, the larger of the two (or the first if they have the same size).

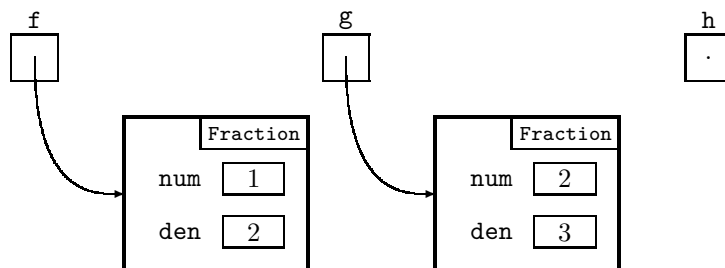
```
public Fraction larger (Fraction other)
{
    if (this.size() >= other.size())
        return this;
    else
        return other;
}
```

If `f`, `g`, and `h` are all of type `Fraction`, then we could use the `larger` method by writing a statement such as

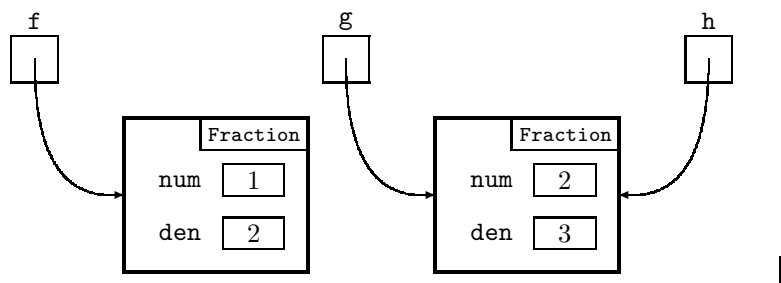
```
h = f.larger(g);
```

This would assign to `h` a reference to the larger when we compare `f` to `g` (or to `f` if `f` and `g` are the same size).

Suppose that, before this call, we had the following situation:



Then, after the call, **h** would also refer to **g**, because $\frac{2}{3} > \frac{1}{2}$. Pictorially, we would have:



Note the use of the reserved word **this** in the example. As we have said previously, **this** refers to the implicit object associated with an instance method. In the call **f.larger(g)**, **this** would refer to **f** while the explicit parameter, **other**, would refer to **g**. In the definition of the **larger** method, the first line of the **if** statement could have been written more simply as

```
if (size() >= other.size())
```

Normally a call to an instance method such as **size** requires an object but, if we omit an object reference, Java assumes that we intended to use **this**, the current implicit object reference.

As with any other methods, instance methods can take the form of commands that do not return values. The next example illustrates this.

Example 3

The method **timesEquals** has the same effect (for **Fraction** objects) that the ***=** operator has for primitive numeric types. It assigns to its implicit **Fraction** parameter the product of itself with its explicit **Fraction** parameter.

```

public void timesEquals(Fraction p)
{
    num *= p.num;
    den *= p.den;
}

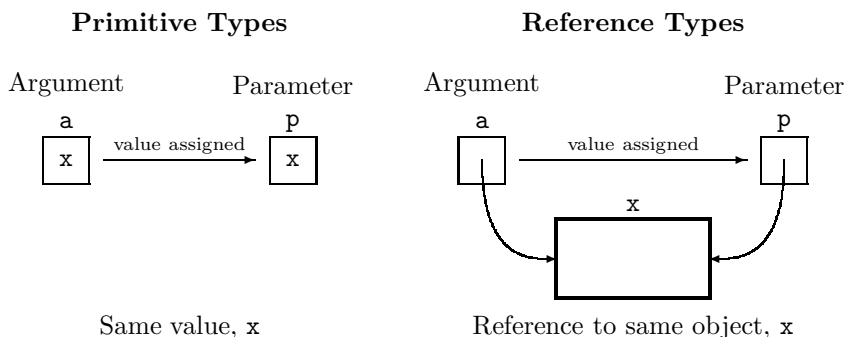
```

If **f** and **g** are objects of type **Fraction**, then the statement

```
f.timesEquals(g);
```

would make **f** represent the product of the values represented by **f** and **g** (while leaving **g** unchanged).

Example 3 also illustrates an important difference between object (reference type) parameters and primitive type parameters. As we have said before, all parameters in Java are passed by value. This means that, at the time of a call to a method, the parameter is assigned the value of the argument. Any subsequent changes to a parameter in a method cannot change the value of the argument in the calling block. For primitive types, the value that is passed to the parameter is the value of the item; for objects, the value that is passed is the value of the *reference* to the object. Thus, for primitive types, the parameter contains an item that has the same value as the corresponding argument; for objects, the parameter refers to the same object as the corresponding argument. This is illustrated in the next diagram.



A consequence of this is that, if a field of an object parameter is altered in a method, then this actually alters the field in the argument's object. In Example 3, the fields of the *implicit* object parameter were altered but the

same principle applies to *explicit* object parameters. In either case, fields of object parameters can be altered in methods.

Our final example illustrates the use of a method that creates an object and returns a reference to that object.

Example 4

The method `times`, when called by the statement

```
Fraction f = g.times(h);
```

where `g` and `h` are both `Fraction` objects, returns a new `Fraction` object equal to the product of `g` and `h` while leaving both `g` and `h` unchanged.

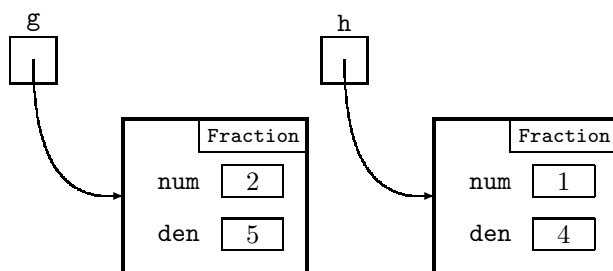
```
public Fraction times (Fraction other)
{
    Fraction result = new Fraction();
    result.num = num * other.num;
    result.den = den * other.den;
    return result;
}
```

In the example, the variable `result` is local to the method but this does not mean that the `Fraction` object to which `result` refers is lost when execution of the method terminates. The statement

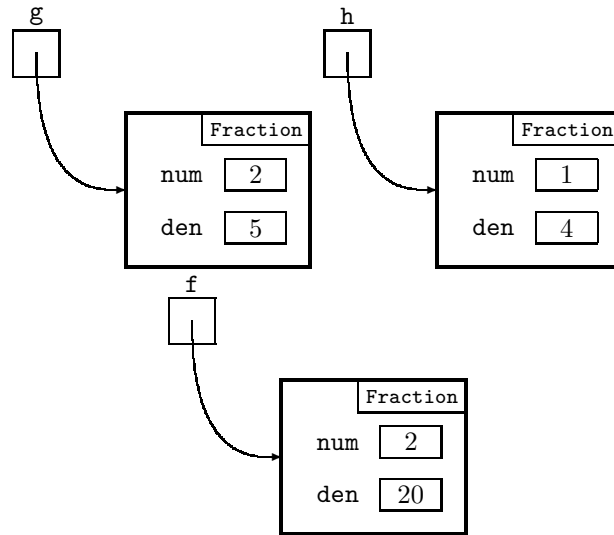
```
Fraction f = g.times(h);
```

returns a reference to the newly created object. This reference is then assigned to `f` so that `f` now refers to the new object, as required.

Once again, we can illustrate the effect of this method with diagrams. Suppose that, before the call to `times`, we had the following:



Then, after execution of the statement `Fraction f = g.times(h);` we would have:



Exercises 6.2

- Suppose that `p`, `q`, and `r` are all objects of type `Fraction`. What fraction would `r` represent after the statement


```
r = p.larger(q);
```

 is executed given that `larger` is the method in Example 2 and
 - `p` represents $\frac{1}{3}$ and `q` represents $\frac{4}{5}$
 - `p` represents $\frac{7}{-5}$ and `q` represents $\frac{-9}{-7}$
 - `p` represents $\frac{5}{6}$ and `q` represents $\frac{-25}{-30}$
 - `p` represents $\frac{-9}{-12}$ and `q` represents $\frac{-3}{4}$
 - `p` represents $\frac{5}{8}$ and `q` represents $\frac{13}{20}$
- Complete the definitions of the following instance methods for the `Fraction` class.
 - `public void plusEquals (Fraction other)`
 The method should have the effect (for `Fraction` objects) that

the `+=` operator has for primitive numeric types. Thus, if called by the statement

```
p.plusEquals(q);
```

(where `p` and `q` are objects of type `Fraction`), the method would make `p` represent the sum of the fractions currently represented by `p` and `q` while the value of `q` would be left unchanged.

- (b) `public Fraction plus (Fraction f)`

The method should return a `Fraction` object whose value is the sum of the implicit object parameter and the explicit parameter, `f`. The method should leave both its explicit and implicit parameters unchanged.

- (c) `public void reduce ()`

The method should reduce its implicit `Fraction` parameter to lowest terms. For example, if `f` represents the fraction $\frac{16}{24}$, the statement

```
f.reduce();
```

should change `f` so that it represents the fraction $\frac{2}{3}$.

3. Suppose that a class `Complex` has been defined as follows:

```
class Complex
{
    double re;
    double im;
}
```

- (a) Write an instance method `modulus` for this class that could be called by a statement like

```
double size = z.modulus();
```

where `z` is of type `Complex`. If `z` represented the value $a + ib$, then the call would set the variable `size` to the value of $|z| = \sqrt{a^2 + b^2}$.

- (b) Write an instance method called `scale` for the class `Complex` that could be called by a statement like

```
z.scale(x);
```

where `z` is of type `Complex` and `x` is a `double` value. If, before the call, `z` represented the value $a + ib$, then, after the call, it should represent the value $x(a + ib)$.

4. Assuming that `z1`, `z2`, and `z3` are of the type `Complex` described in the previous question,

- (a) write an instance method `plus` that, if called by the statement
`z1 = z2.plus(z3);`
would set `z1` to the sum of `z2` and `z3`,
- (b) write an instance method `times` that, if called by the statement
`z1 = z2.times(z3);`
would set `z1` to the product of `z2` and `z3`.

5. Consider again the class `Circle`.

```
class Circle
{
    double x;    // x-coordinate of centre
    double y;    // y-coordinate of centre
    double r;    // radius
}
```

- (a) Write an instance method `area` that returns, as a `double` value, the area of its implicit `Circle` object.
- (b) Write a method `smaller` that could be called by a statement like
`c3 = c1.smaller(c2);`
where `c1`, `c2`, and `c3` are objects of type `Circle`. The method should make `c3` refer to the smaller of the circles represented by `c1` and `c2` (or `c1` if `c1` and `c2` are the same size).
- (c) Write a boolean-valued instance method `isInside` that could be called by a statement like
`boolean contained = c1.isInside(c2);`
The method should return `true` if `c1` is entirely inside `c2` and return `false` otherwise.

6. Write a `main` method that uses the `Circle` class developed in the previous question. The `main` method should perform the following actions.

- (a) Create two `Circle` objects `c1`, representing the circle with centre $(4, -1)$ and radius 3, and `c2`, representing the circle with centre $(3, -2)$ and radius 5.
- (b) Find and print the area of `c1`.

- (c) Determine the smaller of `c1` and `c2` and then print its centre and radius.
- (d) Determine whether or not `c2` lies entirely within `c1` and print an appropriate statement.

6.3 Constructors

In creating objects of the type `Fraction`, we have been using statements that have a form like the following.

```
Fraction f = new Fraction();
```

The parentheses in the expression `Fraction()` make it look like a method call and, in fact, it is. The method is called a *constructor* method. We did not create such a method but, for every class, Java automatically creates a default constructor method that has the same identifier as the class.

Constructor methods are instance methods used to initialize objects at the time the objects are created. The default method initializes any numeric fields to zero, `boolean` fields to `false`, and reference fields to the value `null`. We can, however, write our own constructors that do whatever initialization we wish. As an example, we can use a constructor to initialize objects of type `Fraction` as follows:

Example 1

Suppose, in the class `Fraction`, we created the following constructor:

```
public Fraction (int n, int d)
{
    num = n;
    den = d;
}
```

Then, in our `main` method, say, we could write

```
Fraction f = new Fraction(2,3);
```

This would declare `f` to be of type `Fraction`, create an object of type `Fraction`, set `f` to refer to that object, and initialize the object to represent the fraction $\frac{2}{3}$.

There are a number of things about constructors that the example displays.

1. The identifier of the constructor method is exactly the same as the identifier of the class; both are called **Fraction**.
2. Unlike every method that we have seen so far, the constructor method does not specify a return type (not even **void**). It is implicit that the method returns an object of type **Fraction**.
3. The constructor is an instance method. Thus, it can refer to the instance fields of the class (**num** and **den**), just as any other instance method can.
4. The call to the constructor method differs from other calls to instance methods in that it does not use the dot notation. We wrote **f = new Fraction(2,3);** rather than **new f.Fraction(2,3);**
5. The parameters to the constructor method were called **n** and **d** so that their identifiers did not conflict with **num** and **den**, the identifiers of the object that is implicitly associated with the method. If we had wanted to use **num** and **den** as identifiers of the explicit parameters, we could have written the definition as follows:

```
public Fraction (int num, int den)
{
    this.num = num;
    this.den = den;
}
```

As usual, **this** means the current implicit object. The identifiers on the left side of the assignment statements refer to the **this** object's **num** and **den** fields while the identifiers on the right side refer to the explicit parameters to the method.

Java does not restrict us to having only one constructor method for a class. Like any other methods, constructors can be overloaded with each version having its own signature and each doing some different kind of initialization.

Example 2

The following constructor for the `Fraction` class could be used to create a `Fraction` object that has the same values as an existing `Fraction` object.

```
public Fraction (Fraction f)
{
    num = f.num;
    den = f.den;
}
```

We can even replace the default constructor with one that does the kind of initialization that we want. For our `Fraction` class, the default constructor will, given the call

```
Fraction f = new Fraction();
```

assign both the `num` and `den` fields the value zero, so that the resulting object represents the indeterminate fraction $\frac{0}{0}$. If we wanted to, we could replace this by a constructor that creates a fraction that represents $\frac{0}{1}$.

Example 3

A constructor that initializes `Fraction` objects to represent $\frac{0}{1}$ would take the following form:

```
public Fraction()
{
    num = 0;
    den = 1;
}
```

This constructor could be invoked by the call

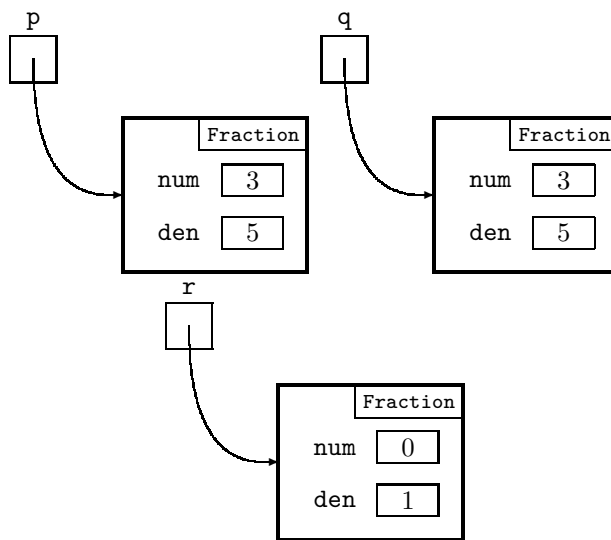
```
Fraction f = new Fraction();
```

just as the default constructor is called.

To show how these methods could be used, consider the following fragment:

```
Fraction p = new Fraction(3,5);
Fraction q = new Fraction(p);
Fraction r = new Fraction();
```

The first statement uses our first constructor to create an object representing the fraction $\frac{3}{5}$. The second statement creates another object that also represents the fraction $\frac{3}{5}$. The third statement creates an object that represents the fraction $\frac{0}{1}$. The diagram illustrates the results.



We said, at the start of this section, that Java supplies a default constructor if we do not write our own. However, if we *do* create our own constructor(s), the default no longer operates. In such a situation, if we want to have a constructor that has no parameters and does only basic initialization, we must write it ourselves.

Example 4

The constructor

```

public Fraction ()
{
    num = den = 0;
}

```

performs the actions of the default constructor for the `Fraction` class.

If a class has multiple constructors, it is sometimes possible to use the code from one constructor in another. The next example illustrates this idea.

Example 5

Suppose that a `Student` class has fields for a name, student number, and number of credits. Normally, a new student will not have any credits. The constructor for such a student might take the form

```
public Student (String n, String sn)
{
    name = n;
    studentNumber = sn;
    credits = 0;
}
```

Sometimes a new student will already have some credits from another school. To handle such cases, we could write another constructor.

```
public Student (String n, String sn, int c)
{
    name = n;
    studentNumber = sn;
    credits = c;
}
```

Notice that the two constructors have two lines of code that are identical. It is usually a good idea to avoid such repetition of multiple lines of code because, if a mistake is found or if the program is being modified at a later time, then the code must be changed in more than one place. We can avoid such repetition here by rewriting the first constructor as follows.

```
public Student (String n, String sn)
{
    this(n,sn,0);
}
```



In Example 5, we have used `this` in a new way, as a reference to the constructor of the current implicit object. If a constructor contains a call to another constructor, that call must be the first statement of the constructor.

Exercises 6.3

1. (a) Extend the definition of the `Fraction` class that we have been developing to include the constructors discussed in this section.
(b) Write a `main` method that first constructs two `Fraction` objects representing the fractions $\frac{5}{7}$ and $\frac{3}{8}$ and then creates two other `Fraction` objects whose values are the sum and product of the original fractions. The method should produce no output; we will deal with printing of objects shortly. Use the methods `plus` and `times` from Section 6.2 in your method.
2. For the class `Circle` defined as follows:

```
class Circle
{
    double x;    // x-coordinate of centre
    double y;    // y-coordinate of centre
    double r;    // radius
}
```

- (a) Write a constructor method that has no parameters. The method should construct a `Circle` object with centre $(0, 0)$ and radius 1.
- (b) Write a constructor method that has three parameters representing the coordinates of the centre and the radius of the object to be constructed.
- (c) Write a constructor method with a parameter, an object of type `Circle`. The method should construct a new `Circle` object with the same field values as those of the parameter.

6.4 Hiding Information

Until now, we have been declaring all the fields of our classes in a form similar to that used for local variables in methods. It is possible, however, to declare fields with visibility modifiers, just as we have seen for methods. If we were to declare a field with the modifier `public`, then that field would be openly available to be read and altered from within any class in any package. By omitting the `public` modifier, we have given the fields *package visibility* — available for inspection and alteration from within any class in the package in which the class is defined. If we have not created a package for our classes, then package visibility is equivalent to directory visibility. That means that omission of any visibility modifier makes the field available from anywhere in the current directory.

Frequently, however, we do not want to give direct access to fields of a class from *any* point outside the class. To illustrate this, let us consider once again our `Fraction` class. In mathematics, fractions are almost always written with positive denominators; fractions of the form $\frac{a}{-b}$ are usually rewritten as $\frac{-a}{b}$ while fractions of the form $\frac{-a}{-b}$ are simplified to $\frac{a}{b}$. Suppose that we decide to maintain the objects of our `Fraction` class in this way. We could start by writing a constructor that creates fractions in the desired form.

Example 1

The following constructor method for the `Fraction` class takes two `int` parameters, the numerator and denominator of a fraction. The constructor ensures that the resulting fraction is represented with a positive denominator.

```
public Fraction (int n, int d)
{
    if (d == 0)
        throw new RuntimeException
            ("Attempt to construct Fraction n/0");
    else if (d < 0)
    {
        num = -n;
        den = -d;
    }
}
```

```
    else
    {
        num = n;
        den = d;
    }
}
```

Notice that a denominator of zero is considered to be an error because the resulting fraction is undefined. If this occurs, the constructor throws an exception. For now, if this exception were to occur during execution of a program, it would print the indicated message and execution of the program would terminate. Exceptions and exception handling are discussed in Appendix E.

If we want to make sure that **Fraction** objects stay in this form, we cannot allow everybody to have direct access to the **num** and **den** fields. To control access to the fields, we can declare them as shown in the next fragment:

```
private int num;
private int den;
```

Now the fields cannot be seen or altered from outside the **Fraction** class. Since the constructor method is inside the **Fraction** class, it can change the values of **private** fields within that class.

If we make the fields of our **Fraction** class **private**, it is not only impossible for outsiders to make direct changes to these fields, it is also impossible to determine directly, from outside the class, what values are stored in the fields. If we wish to permit **private** fields of a class to be seen from outside the class, we can again use methods.

Example 2

The following method allows a user outside the **Fraction** class to access the value of the **num** field of a **Fraction** object.

```
public int getNumerator ()
{
    return num;
}
```

To use this method to obtain the value of the `num` field of a `Fraction` object `f`, we could write

```
int n = f.getNumerator();
```

Methods like the one in Example 2 that allow us to gain access to the values in `private` fields are, unsurprisingly, called *accessor methods*. The identifier that we gave to the accessor method in Example 2 illustrates a commonly used form; in Java, an accessor method for a field named `stuff` is usually called `getStuff`.

Now, if we are to declare fields as `private`, what do we do if we want to modify them from outside the class in which they are defined? The answer is to write methods that can modify the fields for us, in carefully controlled ways. Such methods are sometimes called *mutator methods* because they alter or mutate the values of fields. Customarily, in Java, a method used to set the value of a field named `stuff` would be called `setStuff`.

The process of creating programs so that information within a class is made inaccessible from outside the class is called *encapsulation*. We will, in the coming chapters, not only encapsulate data but also methods. In doing so, we should be able to make our programs more reliable because the data and methods in a class can be tested thoroughly with each other and then only communicate with the outside world in very carefully controlled ways that cannot (if we have done our job well) introduce incorrect results into the class.

Example 3

The method `invert` replaces a `Fraction` object by its inverse while maintaining the positive denominator property.

```
public void invert ()
{
    if (num == 0)
        throw new RuntimeException
            ("Attempt to invert Fraction with value zero");
    else
    {
        int temp = num;
        num = den;
        den = temp;
        if (den < 0)
        {
```

```

        den = -den;
        num = -num;
    }
}

```

Although it may appear to be something of a nuisance to have to go through channels like accessor and mutator methods, these devices do usually pay dividends in the end. We may not have convinced you yet, but we will try to do so as we proceed.

Exercises 6.4

1. Explain the difference between an accessor method and a mutator method.
2. Study the following fragment and then answer the question that follows it. (The dots indicate code that we have omitted.)

```

class Sample
{
    private int i;

    public void foo (Sample s)
    {
        ...
        s.i = 5;  // ***
        ...
    }
}

```

The statement marked with the asterisks attempts to directly alter the value of a **private** field. State, with reasons, whether or not the statement is valid.

3. Write a constructor method for the **Circle** class discussed earlier in this chapter. The method should take three **double** parameters, the coordinates of the centre and the radius. The method should ensure

that the circle's radius is not negative by changing the sign of any negative radius parameters.

6.5 Comparing and Displaying Objects

As we have already seen with assignment statements, operations on objects can sometimes have results that are very different from the corresponding operations with primitive types. In this section, we will explore some other operations with objects.

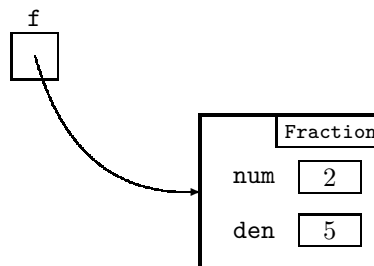
Before looking at new operations, let us recall the way that assignment statements work with reference variables.

Example 1

Suppose, once again, that we are using our `Fraction` class and that we have created an object `f` by writing

```
Fraction f = new Fraction(2,5);
```

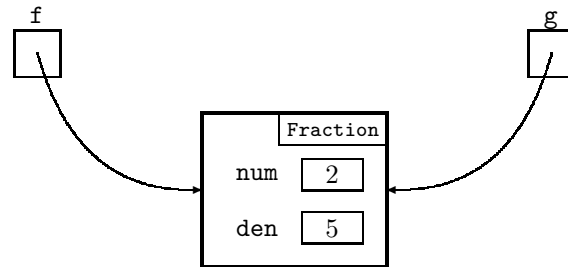
to create the object shown in the next diagram



If we now write

```
Fraction g = f;
```

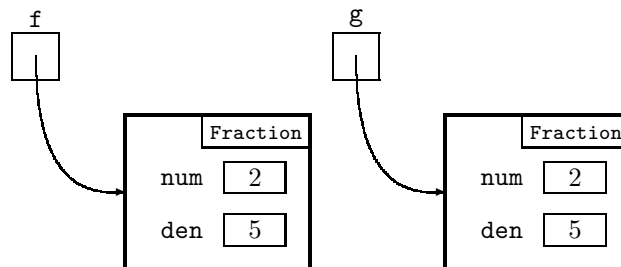
this copies the value from the reference `f` to the reference `g`, producing the result shown in the next diagram where `f` and `g` refer to the same object.



If we wanted `f` and `g` to refer to different objects that represented the same fraction, we would have had to proceed somewhat differently.

Example 2

If we first create `f` as we did in the previous example, and then wrote `Fraction g = new Fraction(f);` this would create a new object whose instance fields have the same values as those of the original. (We are assuming here that we have written an appropriate constructor, as shown in Example 2 of Section 6.3.) The result of this statement is shown in the next diagram.



In this situation, if we were to write

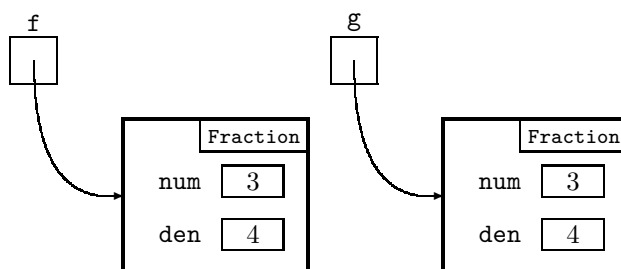
```
g.setNum(1);
```

this would change the `num` field of the object to which `g` refers. Since `f` now refers to a different object, that object would not be changed.

In comparing objects, we must again be conscious of the fact that object variables are references.

Example 3

If `f` and `g` are as shown in the diagram,



then the expression `f == g` would have the value `false` because `f` and `g` refer to distinct objects stored in different locations in memory. The fact that each of those objects represent the same fraction is irrelevant.

If we want to perform a comparison of two objects based on the contents of their fields, we usually do what we have been doing with strings — use a boolean-valued instance method called `equals`. To be consistent with the `equals` methods in the classes of Java’s API, any `equals` method that we write should be an instance method with one explicit parameter that returns `false` if the explicit parameter has the value `null`.

Example 4

The following method will return `true` if and only if two `Fraction` objects have identical fields. The method first checks that the explicit parameter object is not `null`.² If it is not, the method then checks that each of the fields of the two objects are equal.

²There is no need to check that the implicit object is not `null` as Java requires that an instance method be given an instance of an object as an implicit parameter and `null` is not considered to be an instance of an object. Any attempt to use `null` as the value of an instance variable would produce an error message.

```
public boolean equals (Fraction other)
{
    if (other!=null && num==other.num && den==other.den)
        return true;
    else
        return false;
}
```

If you look carefully at this method, you will notice that it returns **true** when the boolean expression that controls the **if** statement is **true** and it returns **false** when that expression is **false**. We can take advantage of this observation to shorten the method definition by simply returning the value of the expression.

```
public boolean equals (Fraction other)
{
    return other!=null && num==other.num && den==other.den;
}
```

We could use this method in contexts like the following:

```
if (p.equals(q)) ...
```



An **equals** method need not require that all fields be equal. The method can apply whatever criteria we choose to consider for equality. For objects of the **Fraction** class, for example, we might consider that two objects are equal if the ratios of the **num** and **den** fields of the objects are equal.

If we do not write our own **equals** method, Java supplies a default version for us, for any type of object that we define. Unfortunately, Java's default **equals** method is fairly useless as it only uses **==** as its criterion for equality. To get something more useful, we must *override* the default method by writing our own as we have in Example 4.

To display values, we have been using the methods **print** and **println**. These methods automatically convert primitive values (like **int** or **double**) to **String** values for printing. If we want to display objects, we can do so in the same way. For any class, Java automatically calls an instance method **toString** provided for this purpose.

Example 5

Consider the following `main` method:

```
public static void main (String[] args)
{
    Fraction f = new Fraction(2,3);
    System.out.println(f);
}
```

On the computer on which this book was written, on the day on which this section was written, this method produced the output

`Fraction@1cc7c5`

The `toString` method that Java provides as a default returns a string that contains the identifier of the class together with a memory reference to the current object. The memory reference may vary from one machine to another and from one day to another on the same machine. As we did with the default for the `equals` method, we can override the default `toString` method with our own.

Example 6

Suppose we add the following method to our `Fraction` class.

```
public String toString ()
{
    return num + "/" + den;
}
```

Now, if we were to run the program in the previous example, Java would use *our* `toString` method rather than the default. The output from the program would be

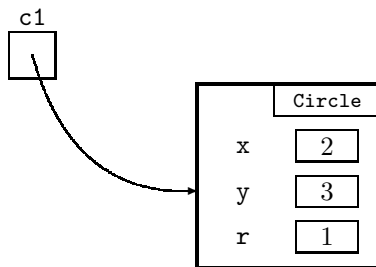
`2/3`

the numerator and denominator of the object in a form that looks like a fraction.

For each class that you create, you should write a `toString` method that overrides the default method. Even if you don't plan to use such methods in your programs, they can be very useful for debugging while you are developing a program involving objects. For each case, choose a form for the string that makes the information clear.

Exercises 6.5

1. If `p` and `q` are both variables of type `Fraction`, under what circumstances will the expression `p == q` have the value `true`?
2. Suppose that `p` and `q` are both of type `Fraction` with `p` representing $\frac{2}{3}$ and `q` representing $\frac{1}{6}$.
 - (a) Draw a diagram like those shown in the text to illustrate this situation.
 - (b) If the statement `p = q;` is executed, draw a diagram to illustrate the result.
3. The diagram shows a `Circle` object of the type that we have been using in exercises throughout this chapter.



- (a) Write a fragment to create a new reference, `c2`, to the same object.
- (b) Write a fragment to create a new `Circle` object `c3`, with the same centre and radius as `c1`.
- (c) Draw diagrams to illustrate the results of executing the code in parts (a) and (b).
- (d) What is the value of the expression `c1 == c2`?

- (e) What is the value of the expression `c1 == c3`?
 - (f) Write a `boolean` instance method called `equals` that returns `true` if and only if one `Circle` object is equal to another one.
 - (g) Write a `toString` method for the `Circle` class. For a `Circle` object with `x = 3`, `y = -4`, and `r = 2`, the `toString` method should return a `String` with the value:
`"centre: (3,-4) radius: 2"`.
4. You have been hired by a wicked witch with a strong interest in children. The program that she uses to help her maintain records has a class `Child` that contains the fields
- ```
private int height; // height in cm
private double mass; // mass in kg
```
- The witch wants you to write an `equals` method for the class. The witch considers two `Child` objects to be equal if their heights differ by no more than 2 cm and their masses differ by no more than 0.5 kg.
5. Write a definition of an `equals` method for the `Fraction` class. Your method should return `true` if and only if the `Fraction` objects being compared represent *equivalent* fractions.

## 6.6 Class Methods

Although we usually use instance methods in our dealings with objects, it is possible (and sometimes more appropriate) to use *class* methods — methods that have the same form as those that we encountered in Chapter 5, with the word `static` in their headings.

As an example, suppose we want to create a class method for our `Fraction` class that adds two objects of type `Fraction` and places the sum in a `Fraction`.

**Example 1**

The class `Fraction` can be extended to contain an addition method as follows:

```
class Fraction
{
 private int num;
 private int den;

 public static Fraction sum (Fraction f1, Fraction f2)
 {
 Fraction total = new Fraction();
 total.num = f1.num*f2.den + f2.num*f1.den;
 total.den = f1.den*f2.den;
 return total;
 }
}
```

Suppose now that we want to call this method from within the `main` method of a program. The way that we invoke the method depends on the way in which we organize our program. If we were to put the `main` method in the `Fraction` class, then we would call this method in the same way that we did in Chapter 5 using expressions of the form

<method identifier>(<parameter list>)

For example, assuming that `f`, `g`, and `h` have all been declared to be of type `Fraction` in the `main` method, then the statement

```
f = sum(g,h);
```

would assign to `f` the sum of the current values of `g` and `h`. Notice in the call the lack of an object and a dot preceding the method name. When we call a class method, there is no longer an implicit object parameter; only explicit parameters are possible.

Usually we do *not* organize our programs so that all methods are in the same class. If the `main` method were in a class separate from the `Fraction` class, then a call to any class method in the `Fraction` class would require a different form. Suppose that our `main` method, in a class other than the `Fraction` class, contains three `Fraction` objects: `f`, `g`, and `h`. Now, to invoke `sum` to add `g` and `h` storing the result in `f`, we could write

```
f = Fraction.sum(g,h);
```

Here the expression that is used to invoke the `sum` method has the same form as those used to invoke the class methods in both the `Math` class and the `In` class:

```
<class identifier>.<method identifier>(<parameter list>)
```

The call first directs Java to the correct class and then to the appropriate method within that class.

## Exercises 6.6

1. Consider once again the class `Circle` that we have seen a number of times in this chapter. Each of the following headers could be used in methods that could be placed in the `Circle` class to determine and return the area of a circle.
  - A: `public static double area (Circle c)`
  - B: `public double area ()`
  - (a) Identify the class method and the instance method giving reasons for your choices.
  - (b) Assuming that a `Circle` object called `disc` has been created and initialized in a class outside the `Circle` class, write a statement that could be used to assign to the `double` variable `discArea` the value of the area of `disc` using method A.
  - (c) Repeat the previous part using method B.
  - (d) Complete the definitions of each of the methods.
  - (e) Write a definition of a class method called `areaRatio` that has two parameters, both of type `Circle`. The method should compute and return, as a `double` value, the ratio of the area of the first circle to the second.
2. Write definitions for the following class methods that could be used in the `Fraction` class.
  - (a) The method `product` should have two `Fraction` parameters. It should return a value of type `Fraction`, the product of the parameters passed to it.
  - (b) The method `abs` should have a single `Fraction` parameter. It should return a value of type `Fraction` in which any negative fields in the parameter have been replaced by their absolute values.

- (c) The method `isPositive` should have one `Fraction` parameter. It should return a `boolean` value: `true` if its parameter represents a positive fraction and `false` otherwise.

## 6.7 Class Fields

We have seen that methods can be written as either instance methods or class methods (using the modifier `static`). We have also seen the use of instance fields in classes. Now we complete this part of the picture of Java by discussing *class fields*.

As with class methods, a class field is created by using the modifier `static` in its declaration. The primary difference between a class field and an instance field is that there is only a single copy of a class field but there is a copy of each instance field for every instance of an object that is created from the class. Like instance fields, class fields are initialized automatically.

Class fields are useful for quantities that can be shared by the members of the class. The next example illustrates this.

### Example 1

If we are creating a class `Car`, we might want to have fields to keep track of fuel consumption rate, distance travelled, and price of gasoline. The first two fields will vary from one car to another and should therefore be instance fields. If all cars in the class use the same grade of gasoline, the price will be the same for all cars. Consequently, there is no point in keeping a separate copy of the price of gas for every `Car` object. Instead, we keep it as a class field so that there is only one copy for the entire class. The declarations of these fields might take the following form.

```
class Car
{
 private double consumptionRate;
 private double distance;
 private static double gasPrice;
}
```

We refer to class fields in a manner similar to that used for class methods. Since a class field is not associated with any object, we refer to it from within its class simply by using its identifier. In the previous example, from within the `Car` class we would refer to the class field for the price of gas as `gasPrice`. If a class field is not declared to be `private`, we can refer to it from outside the class using the form

`<class identifier>.<field identifier>`

If the `gasPrice` field in the previous example had been declared to be `public`, we could refer to it from outside the `Car` class as `Car.gasPrice`.

Class fields are often used for constants associated with a class. For example, if we had a class called `Elevator`, we might want to specify the maximum load that could be carried by any elevator. To do so, we might want to write the declaration

```
public static final int CAPACITY = 1800; // max load in kg
```

From within the `Elevator` class, the field could be called `CAPACITY` while outside the `Elevator` class, the field would have to be referred to as `Elevator.CAPACITY`. We have already encountered such class fields — both `Math.PI` and `Math.E` are constant class fields in the `Math` class.

Notice in the declaration that we made the field `public`. This is common with constant fields. We do not need to encapsulate the data to protect it because making it `final` prevents anybody from tampering with the value that we initially assign to it.

## Exercises 6.7

1. What is the difference between the declaration of a class field and that of an instance field?
2. How do we specify that a field's value cannot be altered?
3. State the convention that is used to distinguish constant class field identifiers from variable class field identifiers.
4. Assume that fields in a class have been declared with the `public` modifier. From outside that class, how would we refer to
  - (a) a class field?
  - (b) an instance field?
5. (a) Write an accessor method called `getPrice` for the `gasPrice` field shown in Example 1 of this section.

- (b) Write a mutator method called `setPrice` to assign a new value to the `gasPrice` field.
  - (c) Write a statement that could be used to set a variable `currentPrice` to the value of the `gasPrice` field.
  - (d) Write a statement that could be used to set the `gasPrice` field to the value 72.9.
6. Suppose that a class `Account` is to be created to maintain records of savings accounts in a bank. State, with reasons, which of the following should be class fields and which should be instance fields.
- (a) minimum balance
  - (b) account number
  - (c) current balance
  - (d) interest rate

## 6.8 Putting the Pieces Together

Throughout this chapter, we have been using `Fraction` objects to illustrate our ideas. In this section we show how one might assemble and use the various pieces that we have been developing.

### Example 1

In the `Fraction` class that follows, we implement various constructors, operations for performing arithmetic, a method, `getFraction`, for creating new `Fraction` objects from input supplied by a user, a `toString` method for printing fractions, and accessor methods that allow us to retrieve values of `private` fields. The `size` method gives the magnitude of a fraction while `equals` can be used to compare two fractions. The `reduce` method is used to ensure that all fractions are maintained in reduced form with non-negative denominators. It has been made `private` because it is not needed outside the class; all `Fraction` objects are created and maintained in reduced form within the class.

```
class Fraction
{
 private int num;
 private int den;
```

```
public Fraction ()
{
 // Create an object representing 0/1.
 num = 0;
 den = 1;
}

public Fraction (int n, int d)
{
 // Create an object whose value is n/d.
 num = n;
 den = d;
 this.reduce();
}

public Fraction (Fraction f)
{
 // Create an object identical to the parameter f
 num = f.num;
 den = f.den;
}

public Fraction plus (Fraction other)
{
 // Create and return a Fraction object with value:
 // this + other
 Fraction result = new Fraction(num*other.den
 + den*other.num, den*other.den);
 return result;
}

public Fraction minus (Fraction other)
{
 // Create and return a Fraction object with value:
 // this - other
 Fraction result = new Fraction(num*other.den
 - den*other.num, den*other.den);
 return result;
}
```

```
public Fraction times (Fraction other)
{
 // Create and return a Fraction object with value:
 // this * other
 Fraction result = new Fraction(num*other.num,
 den*other.den);

 return result;
}

public Fraction dividedBy (Fraction other)
{
 // Create and return a Fraction object with value:
 // this / other
 Fraction result = new Fraction(num*other.den,
 den*other.num);

 return result;
}

public static Fraction getFraction (String prompt)
{
 // Prompt the user for the fields of a fraction and
 // return a new Fraction object (in reduced form).
 Fraction result = new Fraction();
 System.out.println(prompt);
 System.out.println("numerator: ");
 result.num = In.getInt();
 System.out.println("denominator: ");
 result.den = In.getInt();
 result.reduce();
 return result;
}

public String toString ()
{
 // Return a string representing the implicit Fraction
 // object. If the object represents an undefined or
 // indeterminate value, return an appropriate string.
 if (den != 0)
 return num + "/" + den;
}
```

```
 else if (num == 0)
 return "NaN";
 else if (num > 0)
 return "Infinity";
 else
 return "-Infinity";
 }

 public int getNumerator ()
 {
 // An accessor method for the numerator field
 return num;
 }

 public int getDenominator ()
 {
 // An accessor method for the denominator field
 return den;
 }

 public boolean equals (Fraction other)
 {
 // Return true if and only if the fields of the implicit
 // Fraction object are identical to those of other
 return other!=null && num==other.num && den==other.den;
 }

 public double size ()
 {
 // Return the magnitude of a Fraction object
 return Math.abs((double)num/den);
 }

 private void reduce ()
 {
 // Reduce the implicit Fraction object to lowest terms
 // and ensure that the denominator is not negative.

 // eliminate negative denominator
 if (den < 0)
```

```

 {
 den *= -1;
 num *= -1;
 }

 // store zero as 0/1
 if (num == 0 && den != 0)
 den = 1;

 // store infinities as 1/0 or -1/0
 if (den == 0 && num != 0)
 if (num > 0)
 num = 1;
 else
 num = -1;

 // eliminate any common factors in num and den fields
 for (int i = Math.min(Math.abs(num),den); i >= 2; i--)
 if (num % i == 0 && den % i == 0)
 {
 num /= i;
 den /= i;
 }
 }
}

```

Once we have created the `Fraction` class, we can then use it in programs that manipulate fractions in an easy and natural way. The next two examples show the use of some of the methods of the `Fraction` class.

## Example 2

The following program prompts the user for two fractions and then finds and prints their sum, difference, product, and quotient.

```

class FractionArithmetic
{
 public static void main (String[] args)

```

```

{
 Fraction first = Fraction.getFraction
 ("Enter the first fraction");
 Fraction second = Fraction.getFraction
 ("Enter the second fraction");
 System.out.println("sum is "
 + first.plus(second));
 System.out.println("difference is "
 + first.minus(second));
 System.out.println("product is "
 + first.times(second));
 System.out.println("quotient is "
 + first.dividedBy(second));
}
}

```

If a user responds to the prompts by supplying fractions whose values are  $\frac{3}{4}$  and  $\frac{5}{8}$ , the program will respond by printing

|               |       |
|---------------|-------|
| sum is        | 11/8  |
| difference is | 1/8   |
| product is    | 15/32 |
| quotient is   | 6/5   |

### Example 3

The following program prompts the user for a value of  $n$  and then finds and prints the value of the expression

$$\sum_{i=1}^n \left( \frac{1}{i} \times \frac{1}{i+1} \right)$$

```

class SeriesSum
{
 public static void main (String[] args)
 {
 System.out.println("Enter a value of n");
 int n = In.getInt();
 }
}

```

```
Fraction sum = new Fraction();
Fraction factor1 = new Fraction(1,1);
Fraction factor2 = new Fraction(1,2);
for (int i = 1; i <= n; i++)
{
 Fraction term = factor1.times(factor2);
 sum = sum.plus(term);
 factor1 = factor2;
 factor2 = new Fraction(1,i+2);
}
System.out.println("Sum of " + n + " terms: " + sum);
}
```

If a user provides input of 10, the program will print

|                          |
|--------------------------|
| Sum of 10 terms is 10/11 |
|--------------------------|

In each of the last two examples, the resulting program contained two classes: the `Fraction` class and another class containing the `main` method. In programs containing more than one class, we can organize the files containing these classes in a number of ways. One possibility is to place each class in a separate file. In this case, the file containing the class `Sample` should be called `Sample.java`. Alternatively, multiple classes can be placed in one file. In this case, exactly one class should contain the `main` method. The file name must refer to that class. For example, if the class containing `main` is called `Driver`, then the file name should be `Driver.java`.

The visibility modifier `public` can be used with classes, as it is with methods and fields. If no visibility modifier precedes a class definition, then the class has package visibility. If no package has been created, this means that the class is visible from within the current directory. On the other hand, if we declare a class to be `public`, then it can be seen from anywhere. If a file contains more than one class, then only one can be declared `public` and the file name must refer to this class. Combining some of these rules, we can see that, if one of the classes of a multi-class file contains a `main` method, then only this class can be declared `public`.

## Exercises 6.8

1. The `equals` method of the `Fraction` class is used to test for equality of `Fraction` objects. Why do we not simply use the `==` operator to make this test?
2. In the `reduce` method of the `Fraction` class, the index of the `for` statement gets *smaller* with each iteration until it gets to two. Would the method work correctly if the index started at two and became *larger* on each iteration? Explain.
3. Write a program that has a `main` method that uses the `Fraction` class to solve the following problem: prompt the user for three fractions,  $a$ ,  $b$ , and  $c$ , compute the value of  $ab - c^2$ , print the result, and print the sum of the squares of the numerator and denominator of the result.
4. Write a program that uses the `Fraction` class to find and print, as fractions in lowest terms, the values of the expression

$$\sum_{i=1}^n \left( \frac{1}{2i-1} - \frac{1}{2i+1} \right)$$

for  $n = 1, 2, \dots, 10$ .

5. Use your results from the program of the previous question to conjecture the value of

$$\sum_{i=1}^{1000} \left( \frac{1}{2i-1} - \frac{1}{2i+1} \right)$$

6. Modify the `toString` method of the `Fraction` class so that, if the magnitude of the numerator is equal to or greater than that of the denominator, the method returns a string that has the form of either an integer, if appropriate, or a mixed fraction. The table shows examples.

| Value of num | Value of den | String Returned |
|--------------|--------------|-----------------|
| 3            | 5            | "3/5"           |
| 13           | 3            | "4 and 1/3"     |
| 20           | 4            | "5"             |
| -11          | 6            | "-1 and 5/6"    |
| -15          | 5            | "-3"            |

## 6.9 Avoiding Errors and Debugging

### Avoiding Errors

1. With the introduction of instance and class fields in this chapter, we have now seen four varieties of “variable” in Java. It is important that you be able to distinguish them and to be aware of the differences in the ways that they are used and behave.
  - **Local Variables** A local variable is declared inside a block in a method. It is not automatically initialized by Java; this is the responsibility of the programmer. Its scope is from the point at which it is declared to the end of the block containing the declaration. Local variables declared in the initialization expression of a `for` statement are defined only within the `for` statement. The life of a local variable begins when its declaration statement is executed and it ends when the program exits the block in which the variable was declared.
  - **Parameters** A parameter is declared in the header of a method. It is automatically given, as an initial value, the value of the corresponding argument in the call to the method. The scope of a parameter is the body of its method. The life of a parameter begins when the program begins executing the parameter’s method and ends when the program returns to the point from which it was called.
  - **Instance Fields** An instance field is declared within a class but outside any method of that class. Instance fields are automatically initialized (numeric fields to zero, boolean fields to `false`, and reference fields to `null`). There is one copy of an instance field for each object (instantiation) of that class. The scope of an instance field depends on any visibility modifier that might be applied to it with `private` fields being visible only in the class in which they are defined. Visibility of a field may be interfered with by a local variable with the same identifier. (See the first item in the Debugging section of this chapter for an explanation

and example.) The lifetime of an instance field is the lifetime of the object with which it is associated.

- **Class Fields** A class field, like an instance field, is declared within a class but outside any method of that class. A class field is also automatically initialized in the same way and has the same visibility rules. A class field is declared with the modifier **static** and there is only one copy of the field for the entire class. The life of a class field matches the life of its class.

2. Remember that there are two parts to the creation of an object: declaration of a variable that acts as a reference to an object of the appropriate type and the creation of the object itself. For example, consider these statements:

```
Fraction f;
f = new Fraction();
```

The first statement creates `f`, a variable that can act as a reference to a `Fraction` object and the second statement invokes a constructor to create a `Fraction` object to which `f` refers. The two operations can be combined into one statement

```
Fraction f = new Fraction();
```

but you should be clear about the two operations that are being performed by this statement.

3. Remember that Java supplies a default constructor only if we have not created *any* constructor of our own. If you write a constructor that has one parameter, then you can no longer use a constructor for that class that has no parameters unless you write one yourself.
4. We noted earlier in this chapter that the `==` operator, when applied to references to objects, only returns `true` if the references are to the *same* object. To compare objects, you should usually use an `equals` method that compares the values of the fields of objects. Be sure to write your own `equals` method to override the default that Java provides as the default only uses `==` as its criterion for equality.
5. Normally, an expression that invokes an instance method takes the form

```
<object identifier>.<method identifier>(<parameter list>)
```

Within an instance method, however, the `<object identifier>` can be omitted. In that case, Java assumes that the associated object is the current implicit object (which we can refer to using `this`). It may

make your intentions clearer to your readers to write such a call using an expression of the form

```
this.<method identifier>(<parameter list>)
```

## Debugging

1. If a method that assigns values to fields of a class does not seem to be working correctly, check to see that you have not declared the fields within the method. If you do so, then the identifiers will be taken to be *local* variables, not the fields of the class. As an example, consider a constructor of the `Fraction` class, to be called by a statement like

```
Fraction f = new Fraction(2,3);
```

A correct way to write the constructor would be

```
public Fraction (int n, int d)
{
 num = n;
 den = d;
}
```

where `num` and `den` are fields of the `Fraction` class. It is a common mistake to write such a constructor as follows.

```
public Fraction (int n, int d)
{
 int num = n;
 int den = d;
}
```

This will not have the desired effect because `num` and `den` are local variables. The fields with the same identifiers will not be assigned the appropriate values; they will both be left with the default value of zero. We say that the local variables `num` and `den` *shadow* the fields with the same identifiers (because the local variables throw the fields into the shadows where we cannot see them).

2. As we suggested earlier, it is a good idea in developing and testing the methods of a class, to write a `toString` method for the class. Once this is done, it makes the task of tracing the activities of other methods much easier.

## Exercises 6.9

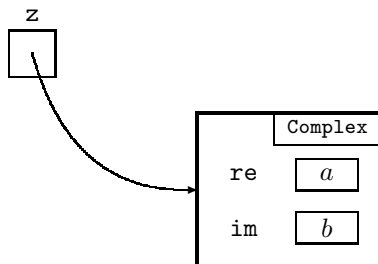
1. Under what circumstances does Java supply a default constructor for a class?
2. Under what circumstances will Java's default `equals` method return the value `true`?
3. The following method for the `Fraction` class that we have been studying throughout this chapter uses the `reduce` method introduced in Section 6.8. Study the method and then answer the questions that follow it.

```
public int crossProduct (Fraction f)
{
 reduce();
 f.reduce();
 return this.num*f.den + this.den*f.num;
}
```

- (a) What object is being reduced in the first call to `reduce`?
- (b) How else could that call have been written?
- (c) How could the `return` statement be shortened?

## 6.10 Review Exercises 6

1. What is the difference between a class and an object?
2. What is the difference between the declaration of a class field and the declaration of an instance field?
3. What is the difference between a call to a class method and a call to an instance method?
4. Suppose that a complex number  $z = a + bi$  is represented by an object of the form shown in the diagram



Write a definition for a class `Complex` that contains

- `private double` fields `re` and `im` for the real and imaginary parts of a number,
- a constructor method that could be called by the statement  
`Complex z = new Complex(2.7,-1.1);`  
 to create an object that represents the complex number  $2.7 - 1.1i$ ,
- a constructor method with no parameters that creates a `Complex` object representing the number 0,
- an instance method that finds the modulus of a complex number,
- an accessor method that returns the value of the real part of a complex number.

5. Suppose that the following fragment is part of a method in the `Complex` class developed in the previous question. Draw diagrams to illustrate the result of executing the fragment.

```

Complex z1, z2, z3;
z1 = new Complex(2,-2);
z2 = new Complex();
z3 = z1;
z2.re = z3.im + 3;
z2.im = 2*z1.im;
z3.im = z1.re / 2;

```

6. Write an instance method `harmonicSum` for the `Fraction` class discussed throughout this chapter. The method should have one `int` parameter, `n`. It should set its implicit parameter to the value of the

fraction found by adding the terms of the harmonic series

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

The fraction should be reduced to lowest terms. If  $n < 1$ , then the fraction should be set to  $\frac{0}{1}$ .

7. Write a class `Lock` that could be used to create electronic lock objects. Each lock may be in either an open (unlocked) or a closed (locked) state and each one is protected by its own integer key which must be used to unlock it. The class should contain the following methods.

- `public Lock (int key)`  
Create a lock that is initially open.
- `public void close ()`  
Close the lock.
- `public boolean isOpen ()`  
Return `true` if and only if the lock is open.
- `public void open (int key)`  
Open the lock if and only if the parameter key matches the lock's own key. If the lock is closed and the keys do not match, count the failed attempt. If the same lock receives three or more failed attempts in a row, print the message "ALARM".

The following `main` method illustrates how the `Lock` class should work.

```
public static void main (String[] args)
{
 Lock lock1 = new Lock(111);
 Lock lock2 = new Lock(222);

 lock1.close();
 lock2.close();

 System.out.println(lock1.isOpen()); // prints false
 lock1.open(123); // fails to open lock1
 lock1.open(456); // fails to open lock1
 lock2.open(222); // opens lock2
 lock1.open(789); // fails - prints ALARM
```

```

 lock1.open(111); // opens lock1
 }

```

## Projects

8. Suppose that rectangles in a Cartesian plane are represented by a class `Rectangle` that has four `private` fields as shown:

```

class Rectangle
{
 private int left; // x-coordinate of left edge
 private int bottom; // y-coordinate of bottom edge
 private int width; // width of rectangle
 private int height // height of rectangle
}

```

- (a) Write a constructor method that has four parameters representing the four fields of the class. The constructor should replace any negative length parameters with zero.
- (b) Write a `toString` method for the `Rectangle` class. For the rectangle with lower left corner located at (3, 2) and having width 4 and height 5, the method should return `"base:(3,2) w:4 h:5"`.
- (c) Write an instance method, `area`, that returns the area of a rectangle.
- (d) Write an instance method, `perimeter`, that returns the perimeter of a rectangle.
- (e) Write a class method, `intersection`, that has two `Rectangle` parameters. The method should return the rectangle formed by the area common to the two rectangles. If they do not intersect, the method should return the rectangle whose instance fields are all zero. If the rectangles touch, the method should return a “rectangle” of zero width or zero length.
- (f) Write a class method, `totalPerimeter`, that has two `Rectangle` parameters. The method should return the total perimeter of the figure formed by the two rectangles. It should only count as part of the perimeter those portions of the edges of a rectangle that are on the exterior of the resulting figure. If one rectangle is completely contained by the other, then the method should